Elec 422/527: VLSI Design I

Tetris Hardware Project

FSM Controller Design Report

Team: Memory Mafia

SanDISK (Sandeep) Ramlochan, Kathryn Files, Atishay RAM Lalgudi

Professor Joseph Cavallaro Spring 2026

1 Introduction and Project Overview

The goal of this project is to design a hardware-level Tetris logic core optimized for a 40-pin Integrated Circuit (IC) environment. The system manages seven standard tetromino shapes within a 16×16 bit-mapped grid (scalable to 8×8). Unlike a software-based game, this design operates as a synchronous digital system that directly manipulates hardware registers to represent the game state. To minimize complexity and gate count, the system avoids high-level video signals (VGA/HDMI) in favor of a row-by-row bit-mapped output, suitable for driving an LED matrix or a terminal-based bit-renderer.

Following Professor Cavallaro's feedback on February 10, 2026, we incorporated the following clarifications:

- **Grid size:** The 16×16 grid is retained but reducible to 8×8 if gate count exceeds the padframe budget. Each cell requires only **1 bit** (occupied vs. empty). A 16×16 grid therefore requires exactly 256 bits of board memory.
- **Display controller:** A secondary display FSM (`DISP_SCAN`) has been integrated into the main FSM as a 16-cycle scan state. It can be factored into a dedicated secondary FSM module if desired.
- **Two-phase clocking:** The FSM uses `clka` and `clkb` as two separate non-overlapping clock inputs, following the `twoP_FSM` template provided on CLEAR. State registers update on `negedge clka` and outputs are driven on `negedge clkb`.

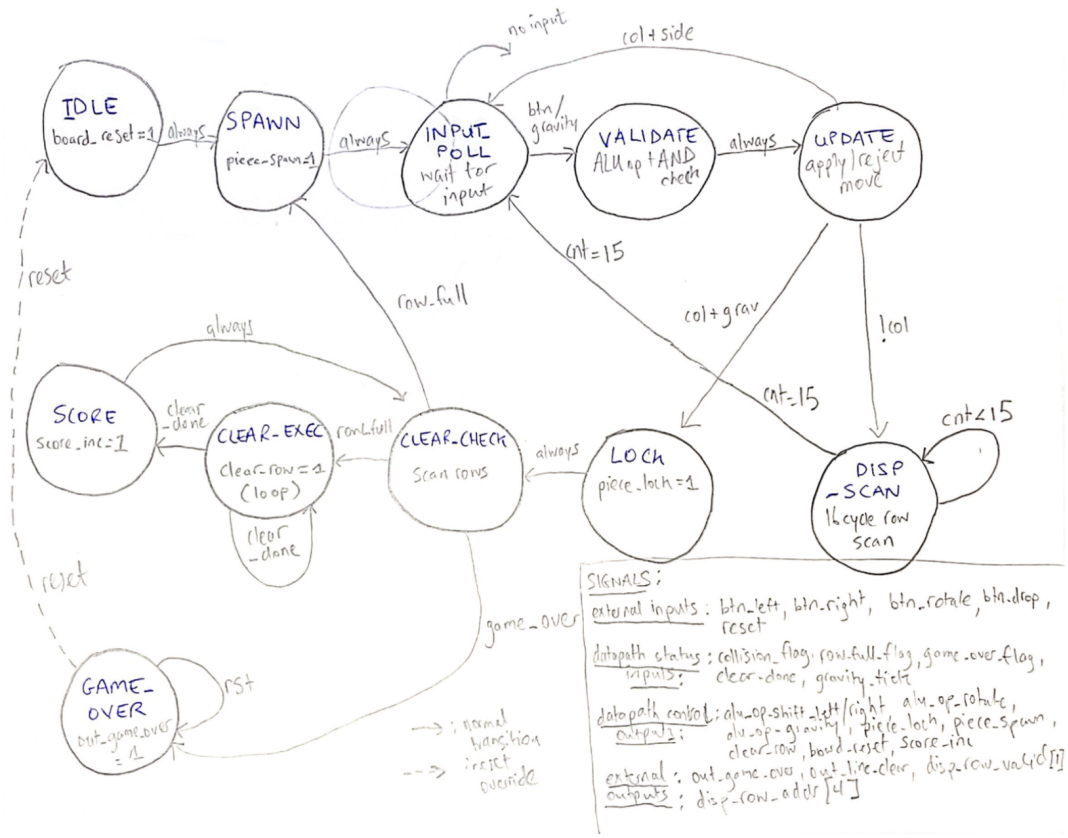

Figure 1: FSM Bubble Diagram

2 Proposed Architecture

1. **FSM Controller** – Drives all datapath control signals.
2. **Datapath (ALU + State Registers)** – Executes shift, rotate, and collision operations.
3. **Grid Memory** (16×16) – 256-bit board representation.
4. **LFSR** – Pseudo-random next-piece selector.
5. **Output Interface** – Row-by-row bit-mapped display bus.

3 Datapath and ALU Design

The datapath is designed with a 16-bit word length, where each word corresponds to a single row of the game board. To maintain a compact chip area, the ALU is restricted to simple operations:

- **Movement:** Horizontal movement (Left/Right) is achieved via logical shift operations on the current piece registers.
- **Collision Detection:** Performed using a bitwise AND between the active piece register and the board memory. If $(Piece \text{ AND } Board) \neq 0$, a collision is flagged.
- **Rotation:** A Look-Up Table (LUT) / ROM stores the four 16-bit orientations for each shape, allowing rotation in a single clock cycle, avoiding complex coordinate transformations.
- **Gravity:** Vertical movement is handled by incrementing a 4-bit Y-coordinate pointer and shifting the piece's row-address in memory.

4 Signal Identification

4.1 External Control Inputs

Signal	Width	Description
`clka`	1	Phase A clock – state register update
`clkb`	1	Phase B clock – output logic update
`restart`	1	Resets FSM to IDLE and clears board
`btn_left`	1	Move active piece left (debounced)
`btn_right`	1	Move active piece right (debounced)
`btn_rotate`	1	Rotate active piece 90° CW via LUT
`btn_drop`	1	Manual hard-drop

Table 1: External Control Inputs

4.2 Datapath Status Inputs (Datapath → FSM)

4.3 Datapath Control Outputs (FSM → Datapath)

4.4 External Output Signals

5 State Assignment

The FSM uses a 4-bit binary encoding with 11 states.

Signal	Width	Description
`collision_flag`	1	Bitwise AND of piece and board; 1 = collision
`row_full_flag`	1	Any row equals `16'hFFFF`
`game_over_flag`	1	Piece locked at top row
`clear_done`	1	Row-shift operation complete
`gravity_tick`	1	Periodic auto-drop pulse

Table 2: Datapath Status Inputs

Signal	Width	Description
`alu_op_shift_left`	1	Shift piece register left
`alu_op_shift_right`	1	Shift piece register right
`alu_op_rotate`	1	Load next rotation from LUT
`alu_op_gravity`	1	Increment Y-pointer (gravity)
`piece_lock`	1	Write piece mask into board memory
`piece_spawn`	1	Load next tetromino into registers
`clear_row`	1	Held HIGH during row clear/shift
`score_inc`	1	One-cycle pulse to increment score
`board_reset`	1	Zero all 256 bits of board memory

Table 3: Datapath Control Outputs

Signal	Width	Description
`out_game_over`	1	Held HIGH in GAME_OVER state
`out_line_clear`	1	Pulsed HIGH during CLEAR_EXEC
`disp_row_valid`	1	HIGH when display bus is valid
`disp_row_addr`	4	Row address 0–15 during DISP_SCAN
`state`	4	FSM state bits for debugging

Table 4: External Output Signals

6 FSM State Descriptions

6.1 IDLE

Entered on power-up or `restart`. Asserts `board_reset` for one cycle to zero all 256 bits of grid memory, then advances unconditionally to SPAWN.

6.2 SPAWN

Asserts `piece_spawn` for one cycle. The datapath loads the next tetromino (selected by LFSR) from a 7-entry ROM into the active piece registers at the initial XY position. Ad-

State	Encoding	Role
IDLE	`4'd0`	Startup; assert `board_reset`
SPAWN	`4'd1`	Load new tetromino
INPUT_POLL	`4'd2`	Wait loop: poll buttons or gravity tick
VALIDATE	`4'd3`	Issue ALU op; collision flag settles
UPDATE	`4'd4`	Three-way branch on collision result
LOCK	`4'd5`	Write piece to board memory
CLEAR_CHECK	`4'd6`	Inspect `row_full_flag` and `game_over_flag`
CLEAR_EXEC	`4'd7`	Hold `clear_row` until `clear_done` (wait loop)
SCORE	`4'd8`	One-cycle `score_inc` pulse
DISP_SCAN	`4'd9`	16-cycle display row scan
GAME_OVER	`4'd10`	Hold `out_game_over` until restart

Table 5: State Encoding Table

vances to INPUT_POLL.

6.3 INPUT_POLL (Wait State)

Loops until at least one of `btn_left`, `btn_right`, `btn_rotate`, `btn_drop`, or `gravity_tick` is asserted. The move command is latched on `negedge clka`. This is the primary idle loop of normal gameplay.

6.4 VALIDATE

Issues one ALU control signal based on the latched move command. The datapath evaluates the bitwise AND of the prospective piece position with board memory, setting `collision_flag` combinationally. Advances to UPDATE.

6.5 UPDATE

Three-way branch:

1. `collision_flag` set and move was gravity/drop ⇒ advance to LOCK.
2. `collision_flag` set and move was side/rotate ⇒ return to INPUT_POLL (move rejected).
3. No collision ⇒ advance to DISP_SCAN.

6.6 LOCK

Asserts `piece_lock` for one cycle. The datapath ORs the active piece register into board memory. Advances to CLEAR_CHECK.

6.7 CLEAR_CHECK

Examines flags: `game_over_flag` ⇒ GAME_OVER; `row_full_flag` ⇒ CLEAR_EXEC; otherwise ⇒ SPAWN.

6.8 CLEAR_EXEC (Multi-Cycle Wait Loop)

`clear_row` and `out_line_clear` are held HIGH until the datapath asserts `clear_done`. The FSM self-loops here for as many cycles as the datapath requires. This is an explicit **internal wait state** to handle variable-duration row shifting.

6.9 SCORE

Asserts `score_inc` for one cycle. Returns to CLEAR_CHECK to handle cascading multi-row clears.

6.10 DISP_SCAN (Secondary Display FSM, 16-Cycle Loop)

A 4-bit internal counter `disp_cnt` cycles 0–15. `disp_row_valid` is held HIGH and `disp_row_addr` is driven by the counter every cycle. After all 16 rows are presented on the output bus, returns to INPUT_POLL.

6.11 GAME_OVER

`out_game_over` is held HIGH indefinitely until `restart` forces a return to IDLE.

7 Main Game Pseudocode FSM

The Main FSM manages the game state and transitions based on user inputs and collision flags:

1. **IDLE/POLLING:** Polls for debounced button signals (Left, Right, Rotate, Drop).
2. **VALIDATE:** Executes bitwise AND between the piece register and board memory. If $(Piece \text{ AND } Board) \neq 0$, a collision is flagged.
3. **UPDATE:** Updates board registers if the move is legal. If a vertical collision occurs, the piece "locks".
4. **CLEAR CHECK:** Scans row registers for the state `16'hFFFF`.
5. **SHIFT:** If a full row is found, it is cleared and higher rows are shifted down.

8 Display Controller FSM

The Display Controller is implemented as a secondary finite state machine (integrated into the main FSM as DISP_SCAN) responsible for continuously scanning the board memory and driving the external display interface.

8.1 FSM States

State	Description
RESET	Initialize row counter
SCAN	Continuously cycle through rows 0–15

8.2 FSM Pseudocode

```
STATE <- RESET

LOOP forever:

    IF reset = 1 THEN
        STATE <- RESET
    ENDIF

    CASE STATE OF

        RESET:
            row_counter <- 0
            STATE <- SCAN

        SCAN:
            IF refresh_tick = 1 THEN
                row_addr <- row_counter
                row_data <- board[row_counter]
                row_counter <- row_counter + 1
                IF row_counter = 16 THEN
                    row_counter <- 0
                ENDIF
            ENDIF
            STATE <- SCAN

    ENDCASE

ENDLOOP
```

This FSM operates independently from the main gameplay controller and ensures continuous display refresh without affecting game logic timing.

9 Memory and Register Budget

To align with the requirement of limited internal storage (target ≈ 128 bits), the design is modular:

- **Game Board:** 16 registers of 16 bits each (256 bits total). Scalable to 8×8 (64 bits) if chip area constraints require.
- **Active Piece State:** 4 registers of 4 bits each to store the current shape mask (16 bits).
- **Control Registers:** 16 bits total for X/Y coordinates, rotation state, and current score.

10 I/O Pin Mapping (40-Pin DIP Constraint)

The design is optimized for a 40-pin dual-in-line package. The pinout is budgeted as follows:

Signal Group	**Pin Count**	**Function**
Power & Clock	4	VDD, GND, System Clock, Reset
User Inputs	4	Rotate, Move Left, Move Right, Manual Drop
Display Data Bus	16	Parallel 16-bit output for the current row
Display Control	4	4-bit Row Address (indicates which row is active)
Status Flags	2	Game Over, Line Clear Pulse
Total	**30 Pins**	**Reserved: 10 pins for testing/expansion**

Table 6: I/O Pin Mapping

11 Two-Phase Clocking

Following the `twoP_FSM` template on CLEAR and the methodology from Homeworks 2 and 3, the FSM has three components:

1. A **combinational function** (`fsm_function`) computes the next state from the current state and all inputs.
2. `always @(negedge clka)` latches `next_state` and internal registers (`move_cmd`, `disp_cnt`).
3. `always @(negedge clkb)` registers the state into `state` and drives all output signals.

The clock relationship follows the irsim non-overlapping pattern:

```
clock clka  0 1 0 0
clock clkb  0 0 0 1
```

Output signal duration summary:

Signal(s)	Duration
`board_reset`, `piece_spawn`, `piece_lock`, `score_inc`	One-cycle pulse
`clear_row`, `out_line_clear`	Multiple cycles (CLEAR_EXEC wait loop)
`disp_row_valid`	16 consecutive cycles (DISP_SCAN loop)
`out_game_over`	Held until restart
`alu_op_*` signals	One-cycle pulse in VALIDATE only

Table 7: Output Signal Duration Summary

12 Simulation Results

12.1 Pre-Synthesis Simulation

The testbench was compiled and simulated using QuestaSim 2023.2 on the CLEAR cluster (node: opal). All 7 tests passed with **0 errors, 1 warning** (the warning is a non-critical optimisation note from vsim):

```
--- TEST 1: Restart ---
  state=1 piece_spawn=1
--- TEST 2: Move left, no collision ---
  alu_op_shift_left=0 state=9
--- TEST 3: Move right, collision rejected ---
  state=2 (expect INPUT_POLL=2)
--- TEST 4: Gravity down collision -> LOCK ---
  piece_lock=1 state=5
```

```
--- TEST 5: Row clear ---
  score_inc=1
--- TEST 6: Game Over ---
  out_game_over=1 (expect 1)
  out_game_over=0 after restart (expect 0)
--- TEST 7: Display scan ---
  disp_row_addr=1..15  valid=1

=== All tests complete ===
Errors: 0, Warnings: 1
```

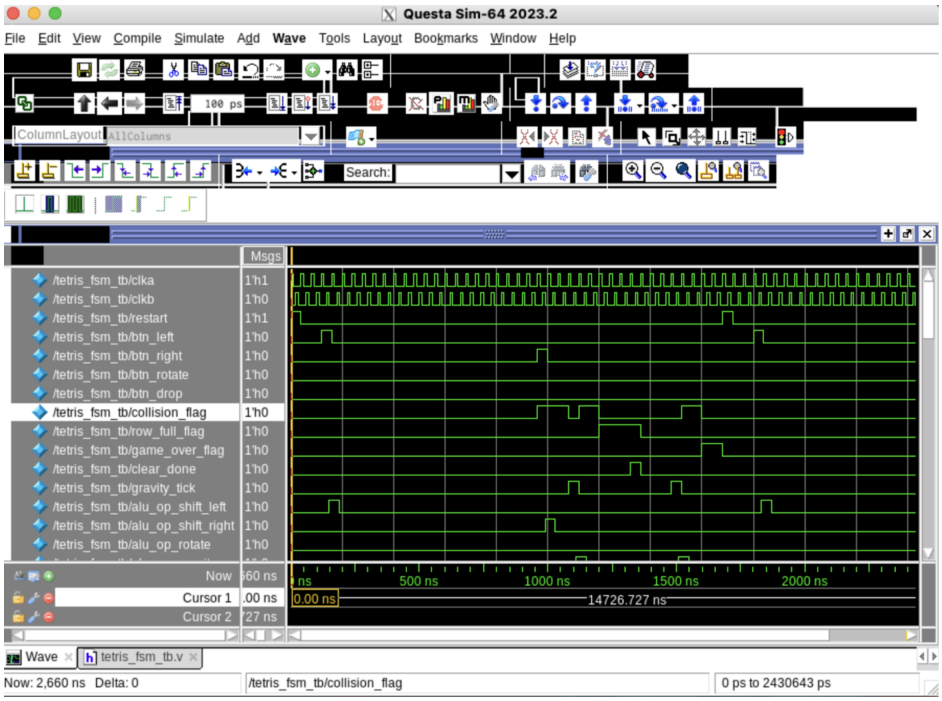

Figure 2: Pre-Synthesis QuestaSim Waveform

12.2 Post-Synthesis Simulation

Post-synthesis simulation will be performed after running Design Compiler with the AMI05 standard cell library. The same testbench will be re-run against the synthesized netlist in the `Questa_post_DC` folder to verify no functional changes were introduced during synthesis.

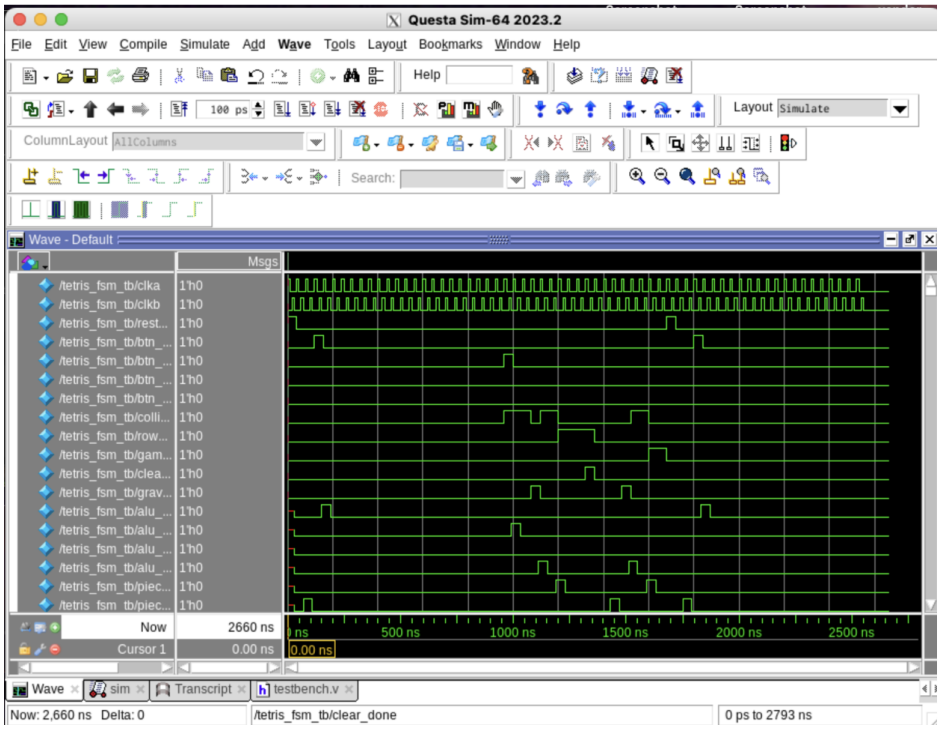

Figure 3: Questa Simulation: Post Synthesis